Sign in

pytorch / pytorch Public

Notifications Fork 23.6k Star 87.9k

Code Issues 5k+ Pull requests 1.1k Actions Projects 12

v2.6.0

pytorch / torch / nn / modules

Go to file

/ linear.py

pytorchmergebot Revert "Validate input types for `torch.nn.Linear` and `torch.nn.Bili…

b6d6aa4 · 6 months ago

293 lines (237 loc) · 10.5 KB .

Code Blame

Raw

```
2   import math
3   from typing import Any
4
5   import torch
6   from torch import Tensor
7   from torch.nn import functional as F, init
8   from torch.nn.parameter import Parameter, UninitializedParameter
9
10  from .lazy import LazyModuleMixin
11  from .module import Module
12
13
```

```python
__all__ = [
    "Bilinear",
    "Identity",
    "LazyLinear",
    "Linear",
]


class Identity(Module):
    r"""A placeholder identity operator that is argument-insensitive.

    Args:
        args: any argument (unused)
        kwargs: any keyword argument (unused)

    Shape:
        - Input: :math:`(*)`, where :math:`*` means any number of dimensions.
        - Output: :math:`(*)`, same shape as the input.

    Examples::

        >>> m = nn.Identity(54, unused_argument1=0.1, unused_argument2=False)
        >>> input = torch.randn(128, 20)
        >>> output = m(input)
        >>> print(output.size())
        torch.Size([128, 20])

    """

    def __init__(self, *args: Any, **kwargs: Any) -> None:
        super().__init__()

    def forward(self, input: Tensor) -> Tensor:
        return input


class Linear(Module):
    r"""Applies an affine linear transformation to the incoming data: :math:`y = xA^T + b`.
```

```
52
53          This module supports :ref:`TensorFloat32<tf32_on_ampere>`.
54
55          On certain ROCm devices, when using float16 inputs this module will use :ref:`different precision<fp16_on_mi200>` for backward.
56
57          Args:
58              in_features: size of each input sample
59              out_features: size of each output sample
60              bias: If set to ``False``, the layer will not learn an additive bias.
61                  Default: ``True``
62
63          Shape:
64              - Input: :math:`(*, H_{in})` where :math:`*` means any number of
65                dimensions including none and :math:`H_{in} = \text{in\_features}`.
66              - Output: :math:`(*, H_{out})` where all but the last dimension
67                are the same shape as the input and :math:`H_{out} = \text{out\_features}`.
68
69          Attributes:
70              weight: the learnable weights of the module of shape
71                  :math:`(\text{out\_features}, \text{in\_features})`. The values are
72                  initialized from :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})`, where
73                  :math:`k = \frac{1}{\text{in\_features}}`
74              bias:   the learnable bias of the module of shape :math:`(\text{out\_features})`.
75                      If :attr:`bias` is ``True``, the values are initialized from
76                      :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})` where
77                      :math:`k = \frac{1}{\text{in\_features}}`
78
79          Examples::
80
81              >>> m = nn.Linear(20, 30)
82              >>> input = torch.randn(128, 20)
83              >>> output = m(input)
84              >>> print(output.size())
85              torch.Size([128, 30])
86          """
87
88          __constants__ = ["in_features", "out_features"]
89          in_features: int
90          out_features: int
```

```
91        weight: Tensor
92
93        def __init__(
94            self,
95            in_features: int,
96            out_features: int,
97            bias: bool = True,
98            device=None,
99            dtype=None,
100       ) -> None:
101           factory_kwargs = {"device": device, "dtype": dtype}
102           super().__init__()
103           self.in_features = in_features
104           self.out_features = out_features
105           self.weight = Parameter(
106               torch.empty((out_features, in_features), **factory_kwargs)
107           )
108           if bias:
109               self.bias = Parameter(torch.empty(out_features, **factory_kwargs))
110           else:
111               self.register_parameter("bias", None)
112           self.reset_parameters()
113
114       def reset_parameters(self) -> None:
115           # Setting a=sqrt(5) in kaiming_uniform is the same as initializing with
116           # uniform(-1/sqrt(in_features), 1/sqrt(in_features)). For details, see
117           # https://github.com/pytorch/pytorch/issues/57109
118           init.kaiming_uniform_(self.weight, a=math.sqrt(5))
119           if self.bias is not None:
120               fan_in, _ = init._calculate_fan_in_and_fan_out(self.weight)
121               bound = 1 / math.sqrt(fan_in) if fan_in > 0 else 0
122               init.uniform_(self.bias, -bound, bound)
123
124       def forward(self, input: Tensor) -> Tensor:
125           return F.linear(input, self.weight, self.bias)
126
127       def extra_repr(self) -> str:
128           return f"in_features={self.in_features}, out_features={self.out_features}, bias={self.bias is not None}"
```

```
129
130
131     # This class exists solely to avoid triggering an obscure error when scripting
132     # an improperly quantized attention layer. See this issue for details:
133     # https://github.com/pytorch/pytorch/issues/58969
134     # TODO: fail fast on quantization API usage error, then remove this class
135     # and replace uses of it with plain Linear
136     class NonDynamicallyQuantizableLinear(Linear):
137         def __init__(
138             self,
139             in_features: int,
140             out_features: int,
141             bias: bool = True,
142             device=None,
143             dtype=None,
144         ) -> None:
145             super().__init__(
146                 in_features, out_features, bias=bias, device=device, dtype=dtype
147             )
148
149
150     class Bilinear(Module):
151         r"""Applies a bilinear transformation to the incoming data: :math:`y = x_1^T A x_2 + b`.
152
153         Args:
154             in1_features: size of each first input sample
155             in2_features: size of each second input sample
156             out_features: size of each output sample
157             bias: If set to False, the layer will not learn an additive bias.
158                 Default: ``True``
159
160         Shape:
161             - Input1: :math:`(*, H_{in1})` where :math:`H_{in1}=\text{in1\_features}` and
162               :math:`*` means any number of additional dimensions including none. All but the last dimension
163               of the inputs should be the same.
164             - Input2: :math:`(*, H_{in2})` where :math:`H_{in2}=\text{in2\_features}`.
165             - Output: :math:`(*, H_{out})` where :math:`H_{out}=\text{out\_features}`
166               and all but the last dimension are the same shape as the input.
```

```
167
168         Attributes:
169             weight: the learnable weights of the module of shape
170                 :math:`(\text{out\_features}, \text{in1\_features}, \text{in2\_features})`.
171                 The values are initialized from :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})`, where
172                 :math:`k = \frac{1}{\text{in1\_features}}`
173             bias:   the learnable bias of the module of shape :math:`(\text{out\_features})`.
174                     If :attr:`bias` is ``True``, the values are initialized from
175                     :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})`, where
176                     :math:`k = \frac{1}{\text{in1\_features}}`
177
178         Examples::
179
180             >>> m = nn.Bilinear(20, 30, 40)
181             >>> input1 = torch.randn(128, 20)
182             >>> input2 = torch.randn(128, 30)
183             >>> output = m(input1, input2)
184             >>> print(output.size())
185             torch.Size([128, 40])
186         """
187
188         __constants__ = ["in1_features", "in2_features", "out_features"]
189         in1_features: int
190         in2_features: int
191         out_features: int
192         weight: Tensor
193
194         def __init__(
195             self,
196             in1_features: int,
197             in2_features: int,
198             out_features: int,
199             bias: bool = True,
200             device=None,
201             dtype=None,
202         ) -> None:
203             factory_kwargs = {"device": device, "dtype": dtype}
204             super().__init__()
205             self.in1_features = in1_features
```

```
206             self.in2_features = in2_features
207             self.out_features = out_features
208             self.weight = Parameter(
209                 torch.empty((out_features, in1_features, in2_features), **factory_kwargs)
210             )
211
212             if bias:
213                 self.bias = Parameter(torch.empty(out_features, **factory_kwargs))
214             else:
215                 self.register_parameter("bias", None)
216             self.reset_parameters()
217
218         def reset_parameters(self) -> None:
219             bound = 1 / math.sqrt(self.weight.size(1))
220             init.uniform_(self.weight, -bound, bound)
221             if self.bias is not None:
222                 init.uniform_(self.bias, -bound, bound)
223
224         def forward(self, input1: Tensor, input2: Tensor) -> Tensor:
225             return F.bilinear(input1, input2, self.weight, self.bias)
226
227         def extra_repr(self) -> str:
228             return (
229                 f"in1_features={self.in1_features}, in2_features={self.in2_features}, "
230                 f"out_features={self.out_features}, bias={self.bias is not None}"
231             )
232
233
234     class LazyLinear(LazyModuleMixin, Linear):
235         r"""A :class:`torch.nn.Linear` module where `in_features` is inferred.
236
237         In this module, the `weight` and `bias` are of :class:`torch.nn.UninitializedParameter`
238         class. They will be initialized after the first call to ``forward`` is done and the
239         module will become a regular :class:`torch.nn.Linear` module. The ``in_features`` argument
240         of the :class:`Linear` is inferred from the ``input.shape[-1]``.
241
242         Check the :class:`torch.nn.modules.lazy.LazyModuleMixin` for further documentation
243         on lazy modules and their limitations.
```

```
244
245         Args:
246             out_features: size of each output sample
247             bias: If set to ``False``, the layer will not learn an additive bias.
248                 Default: ``True``
249
250         Attributes:
251             weight: the learnable weights of the module of shape
252                 :math:`(\text{out\_features}, \text{in\_features})`. The values are
253                 initialized from :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})`, where
254                 :math:`k = \frac{1}{\text{in\_features}}`
255             bias:   the learnable bias of the module of shape :math:`(\text{out\_features})`.
256                     If :attr:`bias` is ``True``, the values are initialized from
257                     :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})` where
258                     :math:`k = \frac{1}{\text{in\_features}}`
259
260
261         """
262
263         cls_to_become = Linear  # type: ignore[assignment]
264         weight: UninitializedParameter
265         bias: UninitializedParameter  # type: ignore[assignment]
266
267         def __init__(
268             self, out_features: int, bias: bool = True, device=None, dtype=None
269         ) -> None:
270             factory_kwargs = {"device": device, "dtype": dtype}
271             # bias is hardcoded to False to avoid creating tensor
272             # that will soon be overwritten.
273             super().__init__(0, 0, False)
274             self.weight = UninitializedParameter(**factory_kwargs)
275             self.out_features = out_features
276             if bias:
277                 self.bias = UninitializedParameter(**factory_kwargs)
278
279         def reset_parameters(self) -> None:
280             if not self.has_uninitialized_params() and self.in_features != 0:
281                 super().reset_parameters()
```

```
282
283     def initialize_parameters(self, input) -> None:  # type: ignore[override]
284         if self.has_uninitialized_params():
285             with torch.no_grad():
286                 self.in_features = input.shape[-1]
287                 self.weight.materialize((self.out_features, self.in_features))
288                 if self.bias is not None:
289                     self.bias.materialize((self.out_features,))
290                 self.reset_parameters()
291
292
293 # TODO: PartialLinear - maybe in sparse?
```